LoRA4VR — Training Log

A living record of experiments, findings, and open questions

What this is. A dated log of experiments run on the LoRA4VR self-play loop. Each entry records what was tried, what happened, and how to reproduce it. Entries are added as experiments complete. Reproduction references a git commit; the code at that commit is the exact code that produced the result.

System. `Qwen2.5-Coder-3B-Instruct` fine-tuned with GRPO/DAPO on a Google Cloud L4 GPU (24 GB). The loop generates `policy(state) -> action` Python code, scores it by running the policy in a sandboxed environment, and trains LoRA adapters on the reward signal.

2026-04-13 Environment design and first training runs

`d96f7cb` · *puck-on-tilted-plane env, runs 1–3*

Motivation

CartPole was the original seed task but it is trivially solved by a two-line bang-bang policy, giving a flat learning curve from iteration 0. Any standard RL benchmark (LunarLander, Pendulum, MountainCar) is heavily represented in the base model's pretraining data, so improvement from LoRA would be confounded with prior knowledge. We need an environment the model has to *read* rather than recall.

The puck environment

A puck moves in a bounded $[-6, 6]^2$ arena under a constant hidden tilt (t_x, t_y) and linear drag μ. The policy receives $\{p_x, p_y, v_x, v_y, g_x, g_y\}$ and must drive the puck to the goal within 120 steps. Hidden parameters — tilt, drag, thrust limit $a_{\max}$, initial velocity — are sampled per scenario from a seed and are not in the state dict. The tight thrust envelope ($a_{\max} = \|tilt\|_2 \times 1.05 + \epsilon$) ensures that a naive "thrust straight at the goal" policy fails because it cannot cancel tilt drift.

Reward: $-\Delta t \cdot \text{dist}(p, g)$ per step; $+50$ bonus on reaching the goal. Errored episodes score -200 per scenario (floor).

Baseline gap (200 seeds, hand-written policies):

- Naive aim: $\approx 26\%$ goal rate
- PD + tilt feedforward: $\approx 73\%$ goal rate

This gap is real headroom for LoRA to exploit.

Bugs encountered and fixed

Three bugs had to be resolved before any learning was possible.

Run 1 — OOM on L4. Default config ($G = 16$, batch 16, max completion 900 tokens) ran out of memory at the backward pass. Fix (PR #6): halved G and batch to 8, trimmed completion length to 600.

Run 2 — Reward sign flip. Training ran but degraded (fewer running samples each iteration). Errored samples scored 0.0 while samples that ran and drifted integrated large negative rewards (≈ -280). GRPO's normalized advantage placed the 0.0 errors *above* the mean, so the trainer rewarded broken code. Fix (PR #7): errored scenarios score -200, making broken code strictly the worst outcome.

Run 3 — Dead advantages. With LR $= 10^{-5}$ and only 1 training prompt (4 gradient steps per outer iteration), $\approx 75\%$ of batches were uniformly errored (advantage $= 0$, no gradient). On the remaining steps, weight deltas were $\sim 10^{-6}$ — indistinguishable from zero. The reward was identical to 7 decimal places at iteration 0 and iteration 9. Fix (PR #10): LR bumped $10\times$ to 10^{-4}; epochs raised to 20 (20 gradient steps per outer iteration on 1 prompt).

Reproduce

Commits: OOM fix `ef228fe`, reward floor `ef7e0c9`, LR/epoch fix `e9353d7`. Env source: `scripts/seed_tasks/puck_tilted_plane_env.py`.

2026-04-18 Run 5 probe: base model vs. 5 iterations of GRPO

`d5bc81c` · *puck_grpo_run_5, iters 0 and 5, $n = 20$ samples each*

Setup

Run 5 used the config at `e9353d7` (LR $= 10^{-4}$, $G = 8$, 20 epochs per outer iteration, DAPO loss, $\beta = 0$). It completed 5 outer iterations before the VM was stopped. Adapter checkpoints were recovered to `models/lora/puck_grpo_run_5/`.

To inspect what the model actually writes, we sample 20 policies from the base model (iter 0, no adapters) and from the trained model (iter 5), score each against 8 scenarios, and compare. The tool is `scripts/probe_generation.py`; it saves per-sample policy files, a summary CSV, and per-scenario reward breakdown to `data/probe/{run_name}_iter{N}_t{temp}/`.

Results

	Ran OK	Crashed	Mean reward (runners)	Best reward
Iter 0 (base)	13 / 20	7 / 20	−218.9	−92.4
Iter 5	20 / 20	0 / 20	−140.4	−140.4

Iter 0

The base model generates varied PD/PID controllers but crashes 35% of the time by attempting to read hidden env internals (`a_max`, `dt`, `steps`) from the state dict. The 13 successful policies span a reward range of $[-92, -373]$ with genuine variation in gain choices. The best policy uses asymmetric gains ($k_{px} = 2.0, k_{dx} = 1.0$; $k_{py} = 1.5, k_{dy} = 1.0$) and scores -92.4 over 8 scenarios.

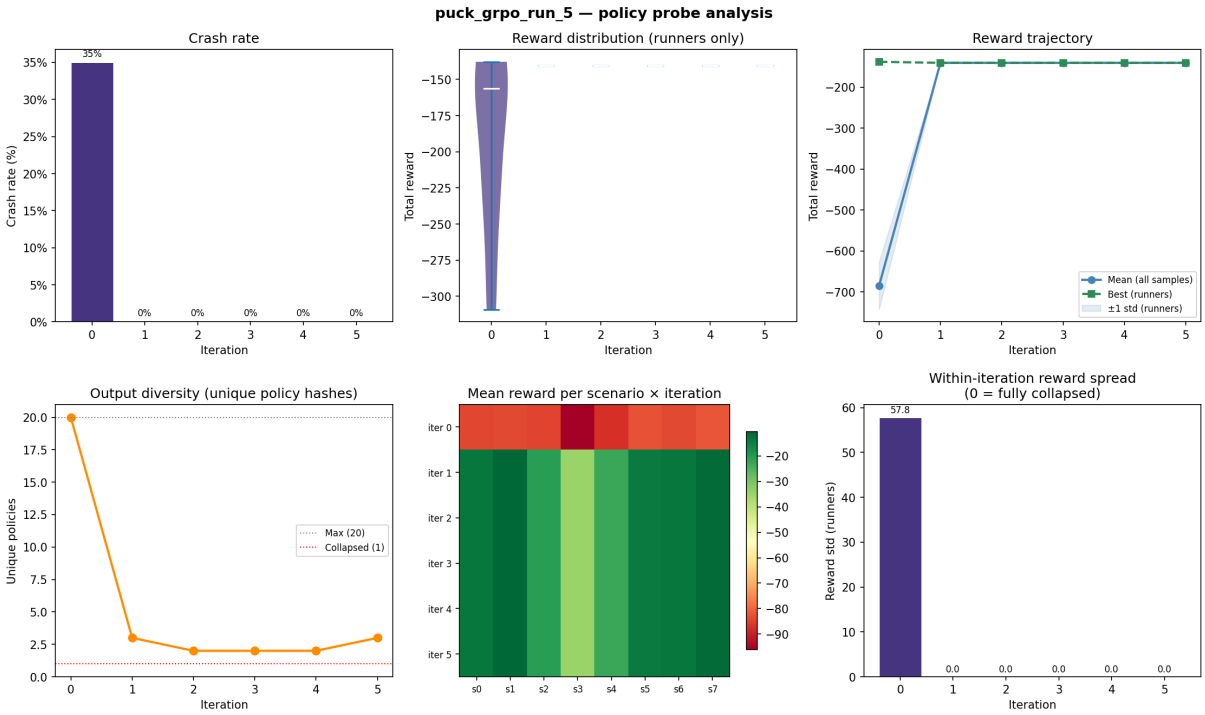


Figure 1: Run 5 probe analysis: base model (iter 0) vs. 5 iterations of GRPO. Generated by `scripts/analyze_probe.py --run_name puck_grpo_run_5`.

Iter 5

Zero crashes. The error-reading habit is gone. However, all 20 samples at temperature 0.8 produce *exactly* −140.3516 (identical to 4 decimal places). Every policy is byte-for-byte the same: $k_p = 2.0$, $k_d = 1.5$. This is the **illustrative example embedded in the prompt template** (`configs/prompt_template.txt`). The model has memorized and reproduced it verbatim.

Interpretation

Training achieved two things and failed at a third.

Achieved. Error rate dropped from 35% to 0% — the model learned not to read hidden env attributes. Mean reward over runners improved by 35%.

Failed. The model collapsed to a single policy. With $\beta = 0$ (no KL penalty), one training prompt, and 20 gradient steps per outer iteration, GRPO found the prompt example as a high-reward fixed point and reinforced it into a delta distribution. By iter 5, temperature 0.8 sampling produces zero diversity. The best reachable policy is now −140.4, worse than the base model's best (−92.4).

This is reward hacking: the model optimized the training distribution (memorize the example) rather than the environment.

Hypotheses to test

1. **Remove or paraphrase the prompt example.** It is the attractor GRPO is collapsing to. A deliberately suboptimal or absent example removes the fixed point.
2. **More training prompts.** With 1 prompt, every outer iteration trains on the same input. Adding 4–8 tasks with varied dynamics forces generalization.

3. **Raise $\beta > 0$.** A non-zero KL coefficient prevents the policy from drifting too far from the base distribution and preserves diversity.
4. **Reduce epochs.** 20 gradient steps on 1 prompt per outer iteration is aggressive. 5–10 may preserve more exploration.
5. **Probe at every intermediate checkpoint.** Does collapse happen at iter 1 or gradually? One probe per checkpoint would pinpoint when diversity dies.

Open questions

- What does the W&B reward curve look like across the 5 iterations? (The run at `puck_grpo_run_5-orchestrator` has data but some post-resume steps may be missing due to a step-monotonicity issue.)
- What is the PD+feedforward ceiling expressed in probe-script reward units? This would establish what score a successful LoRA policy needs to beat.

Reproduce

At commit `d5bc81c`, with adapter checkpoints in `models/lora/puck_grpo_run_5/`:

```
scripts/probe_generation.py --iter 0 --run_name puck_grpo_run_5 --n 20
```

```
scripts/probe_generation.py --iter 5 --run_name puck_grpo_run_5 --n 20
```

Results land in `data/probe/puck_grpo_run_5_iter{000,005}_t0p80/`.

```
scripts/analyze_probe.py --run_name puck_grpo_run_5
```

2026-04-19 Run 8: removing the prompt example unlocks genuine learning

`7ce5526` · *puck_grpo_run_8, 28 outer iterations, iters 0/4/8/12/16/20/24/27 probed ($n = 20$ each)*

Setup

Run 7 (aborted at iter 1) confirmed the model was already generating diverse policies without the prompt example. Run 8 ran the full 30-iteration schedule on the same config as run 5 (`e9353d7`), with the single change that the illustrative $k_p = 2.0$, $k_d = 1.5$ example was removed from `configs/prompt_template.txt` (PR #13, `e5fe162`). Adapters were downloaded from the GCP VM and placed in `models/lora/puck_grpo_run_8/`. Eight evenly-spaced iterations were probed to capture the full learning trajectory without evaluating all 28.

Results

Interpretation

This is the first run showing genuine learning. Three phases are visible. *Iters 0–4*: crashes drop from 85% to 0% as the model learns the valid state-key constraint. *Iters 4–16*: reward improves steadily from −221 to near zero — the model is reaching the goal on some scenarios but not consistently. *Iters 16–20*: a sharp transition to positive reward (+199 mean), indicating consistent goal reaching across most scenarios. The curve flattens after iter 20 with marginal gains through iter 27.

The single change responsible is the removal of the prompt example. In run 5 the example provided a ready-made fixed point that GRPO collapsed to after one update. Without it, the model must explore the policy space, and GRPO has real variance to amplify.

Iter	Crashes	Mean reward	Best reward
0	17 / 20	−1403	−223
4	0 / 20	−221	−165
8	0 / 20	−69	−64
12	0 / 20	−67	−54
16	0 / 20	−28	+12
20	0 / 20	+199	+248
24	0 / 20	+212	+247
27	0 / 20	+215	+248

Table 1: 20 samples × 8 scenarios each. Reward > 0 indicates consistent goal reaching (goal bonus = +50 per scenario).

The iter-27 best reward of +248 approaches the hand-written PD+feedforward ceiling (estimated at ≈ +270 for 8 scenarios at +50 goal bonus each, assuming ∼ 73% goal rate from the 200-seed baseline).

Hypotheses / next steps

1. **Probe all 28 iterations** to pinpoint exactly when the phase transition at iter 16–20 occurs and whether it is sharp or gradual.
2. **Measure diversity at convergence.** Run 5 collapsed to 1 unique policy; does run 8 maintain diversity at iter 27, or does it collapse to a different fixed point once reward saturates?
3. **Compare against the hand-written ceiling.** Score the PD+feedforward baseline with `scripts/probe_generation.py` at iter 0 to get a reward number in the same units.
4. **Train longer or with more prompts** to see whether the plateau at iter 20–27 is a capacity limit or simply convergence of the single training task.

Reproduce

At `7ce5526`, with adapters in `models/lora/puck_grpo_run_8/`:

```
scripts/probe_all_iters.py --run_name puck_grpo_run_8 --iters 0 4 8 12 16 20 24 27
--n 20

scripts/analyze_probe.py --run_name puck_grpo_run_8
```

2026-04-22 Policy analysis: what run 8 learned, and why iter 0 failed

`e6bc200` · *manual probe inspection, run 8 iter 0 and iter 27*

Setup

With run 8 showing genuine learning (mean reward +215 at iter 27, up from −1403 at iter 0), the question was: what strategy did the model actually discover, and why was the base model unable to write a working policy at all? All 20 iter-0 and all 20 iter-27 policy files from `data/probe/puck_grpo_run_8_iter{000,027}_t0p80/` were read and compared manually.

Results

Iter 27 — convergent velocity P-controller. Every iter-27 sample uses the same structural template: compute the angle to the goal, set a desired velocity of magnitude *speed* in that

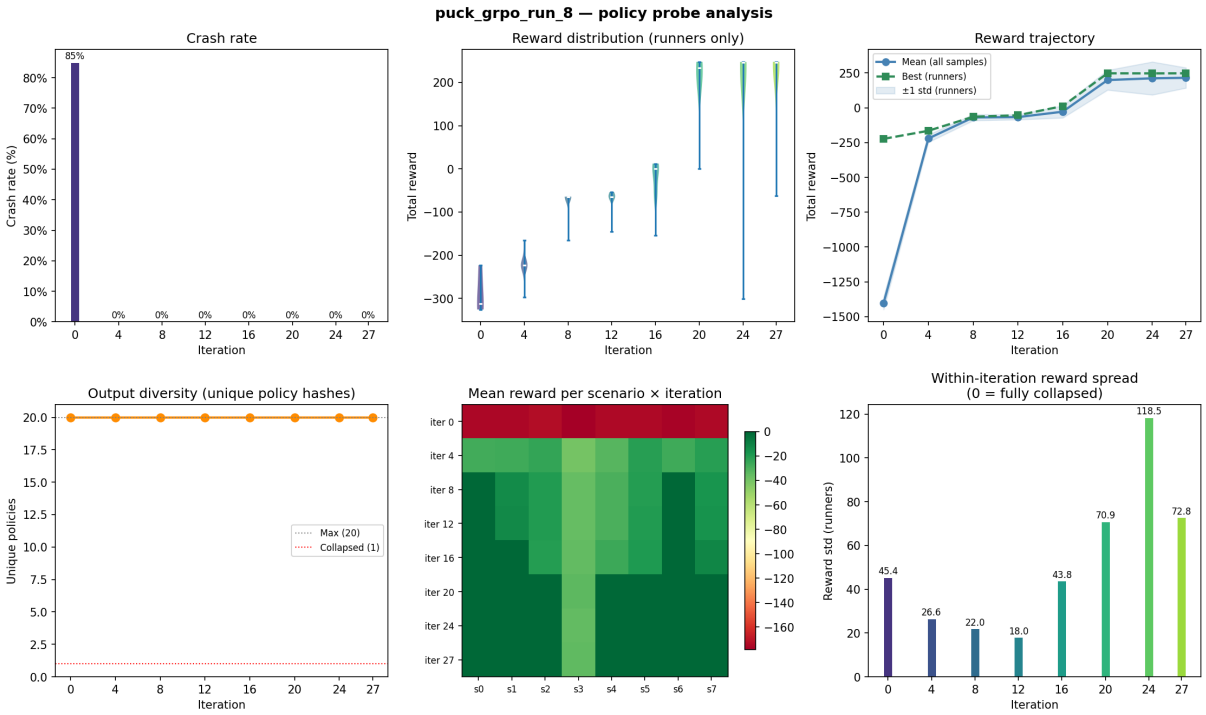


Figure 2: Run 8 learning curve: mean and best reward across 8 probed iterations ($n = 20$ samples each). Generated by `scripts/analyze_probe.py --run_name puck_grpo_run_8`.

direction, and command acceleration proportional to the velocity error:

$$a = \text{gain} \times (v_{\text{desired}} - v_{\text{current}})$$

The only variation across the 20 samples is in two scalar hyperparameters: desired speed (1.0–5.0) and gain ($/0.05 = \times 20$, $\times 10$, $\times 20$). The three below-average samples each have a distinct defect: sample 08 contains a positional-step bug that degenerates to naive aim (reward -62.8); sample 10 sets speed $= 1.0$, too slow to reach the goal in 120 steps; sample 15 self-clips at 3.0 below the env's own $a_{\max}$.

Iter 0 — API mismatch causes 85% crash rate. 17 of 20 base-model policies crashed with KeyError at reward floor -1600. The specific hidden keys accessed are summarised in Table 2. The three survivors avoided hidden keys only by accident: two hardcoded thrust magnitude constants and one hardcoded all physics parameters (`tilt_x = 0.0`, `mu = 0.5`, `a_max = 1.5`). The survivors scored -223, -312, and -325 — all worse than the naive aim baseline.

Hidden key	Samples accessing it
`a_max`	00, 01, 02, 03, 04, 05, 07, 08, 11, 12, 14, 15, 16, 19
`tilt_x`, `tilt_y`	01, 03, 04, 06, 08, 11, 12, 13, 14, 15, 16, 19
`mu`	01, 04, 07, 08, 12, 15, 16
`dt`	16, 17

Table 2: Hidden state keys accessed by crashing iter-0 samples.

Interpretation

What iter 27 discovered. The trained model found a velocity P-controller rather than the PD-on-position + tilt-feedforward strategy used by the hand-written baseline. This is structurally

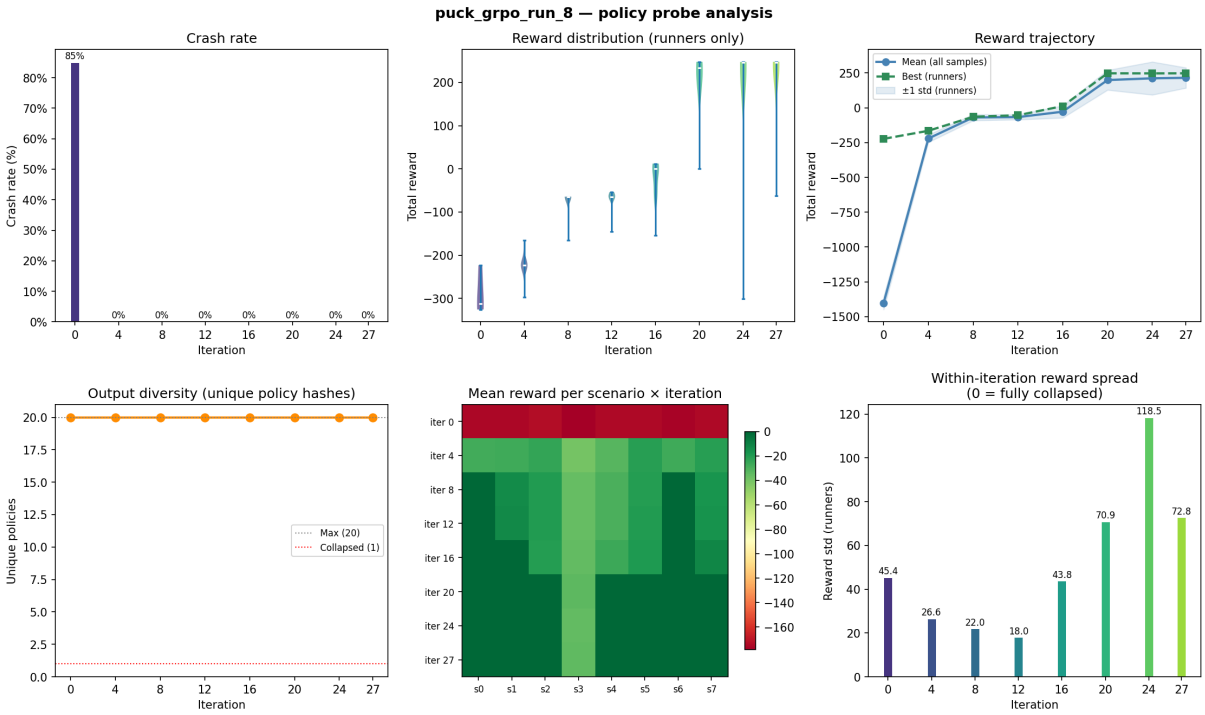


Figure 3: Run 8 learning curve across the 8 probed iterations, reproduced here for context. The phase transition at iter 16–20 is the backdrop for the policy structure analysis above.

different: there is no position error term and no tilt awareness. The approach works because high-gain velocity tracking rejects tilt disturbances reactively within a few timesteps, yielding near-ceiling reward without explicit feedforward. The residual 8% gap to the hand-written ceiling (+248 vs. ≈ +270) is consistent with reactive rather than proactive tilt rejection.

Why iter 0 failed. The base model's prior — drawn from pretraining on standard RL codebases — is that a physics simulation exposes all its parameters through the state dict or environment attributes. 14 of 17 crashing samples attempted tilt compensation or drag cancellation that would have been correct if the hidden parameters were accessible. The model's control-theory knowledge was largely sound; its environment-API assumption was wrong. GRPO fixed the API assumption without teaching better control theory, because velocity tracking was sufficient to saturate the available reward.

Hypotheses / next steps

1. **Probe the transition from position-thrust (iter 0 survivors) to velocity-tracking (iter 27).** At which iteration does the velocity-control template first appear? Checking iter-4 policies would show whether it emerges immediately after crash rate drops.
2. **Test whether harder tilt distributions force tilt compensation.** With wider tilt ranges the reactive velocity-tracker may fail; GRPO would then need to discover an explicit feedforward term to push reward higher.
3. **Provide the hidden parameters in the state dict and retrain.** If the base model already knows how to use tilt/drag/a_max (most iter-0 samples attempt it), a single run with those keys exposed could achieve the hand-written ceiling immediately and reveal what further headroom exists beyond it.

Open questions

- Would the velocity-tracking template generalise to a new environment with different dynamics, or is it a memorised pattern for the puck task?
- The best iter-27 policy (+247.6) and the best iter-0 survivor (−223) both use only valid state keys. Is the iter-27 template a refinement of the iter-0 survivor template, or did it emerge independently under GRPO pressure?

Reproduce

Manual inspection at `e6bc200`; policy files already present in `data/probe/` (run 8, iter 0 and iter 27 probe directories).

2026-04-22 Phase transition iter 16→20: gain and speed jointly determine goal capture

`cd60244` · *manual inspection of iter016 and iter020 probe files, run 8*

Setup

The run-8 learning curve shows a sharp mean-reward jump from +28 (iter 16) to +199 (iter 20). To understand the cause, all 40 policy files from the run-8 iter-16 and iter-20 probe directories were read and compared.

Results

Both iterations use the velocity-tracking template without exception: no crashes, no hidden-key reads. The difference lies entirely in the choice of two scalar hyperparameters.

Iter	Dominant speed	Dominant gain	Mean reward	Best reward
16	10.0	×1	+28	+12
20	2.0–5.0	×10–50	+199	+248

At iter 16, 14 of 20 samples set `desired_speed = 10` and compute acceleration as the raw velocity error (`ax = desired_vx - vx`, gain = 1). Five samples compute the desired velocity as an unnormalized position vector (`desired_vx = speed * dx`), giving a distance-proportional target that oscillates near the goal; these score −53 to −154. By iter 20, all samples use speeds of 1.5–5.0 and gains of ×10–×50 (via `/ 0.05`, `* 10.0`, `* 20.0`, or chained multipliers). The one failure at iter 20 (sample 14, reward +0.5) is the sole sample that reverts to `desired_speed = 10` with gain ×10.

Interpretation

The controller has no explicit distance term and does not decelerate near the goal. Goal capture depends on two mechanisms.

Finite goal-zone radius. The puck need only be inside the goal radius at one timestep. At speed v and $dt = 0.05\,\text{s}$, the per-step displacement is $v \cdot dt$. At 3 m/s this is 0.15 m, comfortably below the goal diameter ($\approx 0.5\,\text{m}$), guaranteeing detection on a direct pass.

Tilt rejection and overshoot amplitude. After the puck passes the goal, `angle_to_goal` flips immediately; the controller commands a reversal. The overshoot amplitude before the reversal takes effect scales as $v_{\text{desired}}/\text{gain}$: low speed and high gain together keep the oscillation amplitude small so the puck re-enters the goal zone on the next pass. More importantly, during the approach,

a lateral tilt acceleration $\mathbf{g}_\perp$ induces a sideways velocity error δv. With gain = 1, the correction is δv (which clips immediately at $a_{\max}$, providing no extra correction). With gain = 20, the correction is $20\,\delta v$ in the linear regime, rejecting tilt drift 20× faster and keeping the trajectory aligned with the goal despite hidden tilt.

The iter-16 failure is therefore not discrete-timestep miss (step size 0.2 m at terminal velocity ≈ 4 m/s is still inside the goal diameter) but poor tilt rejection: with gain = 1 the puck drifts off the goal axis and misses the zone on most scenarios. GRPO resolved this by selectively reinforcing samples where the gain happened to be high enough to overcome tilt drift, receiving the sharp +50 goal bonus as the reward signal.

Hypotheses / next steps

1. **Probe iter 17–19** to determine whether the gain shift happens in one step or gradually. The +50 bonus is a discontinuous signal that could cause a single-iteration jump if the right (speed, gain) combination appears at random and is then strongly reinforced.
2. **Measure whether gain or speed is the primary driver.** Evaluate policies with high gain + high speed, and low gain + low speed, on the 8-scenario test set to separate the tilt-rejection and step-size effects.
3. **Test with wider tilt distributions.** If tilt rejection is the bottleneck, harder tilt scenarios should widen the gap between gain = 1 and gain = 20 controllers, making the transition more pronounced.

Open questions

- The transition occurs at a specific GRPO iteration. Is there a single lucky sample with high gain that crossed the reward threshold and was amplified, or did gain drift upward continuously across iter 16–20?
- What is the minimum gain needed for reliable goal capture at a given tilt magnitude? This would give a quantitative criterion for when the velocity-tracking controller is "good enough."

Reproduce

Manual inspection at `cd60244`; policy files already present in `data/probe/` (run 8, iter 16 and iter 20 probe directories).

2026-04-27 Branched-rollout N=2 OOMs on L4; plan to decompose loss per turn

`8b94cc7` · *branched-N=2 GRPO smoke, PR #16*

Setup / motivation

After run 8 produced a clean single-turn baseline, the next experiment is N=2 multi-turn GRPO with branched MC rollouts (G_1 first-turn variants, G_2 continuations each), using turn-2 mean reward as an empirical $\hat{V}_1$ to give turn-1 a credit-assignment signal without a learned critic. The override (`src/branched_trainer.py`) plugs into TRL 0.28's `GRPOTrainer` by replacing `_generate_and_score_completions` and concatenating each trajectory as [prompt | turn1 | tool | turn2] before handing it to the parent's `_compute_loss`. The smoke goal was an end-to-end GPU run on L4 (24 GB) to validate plumbing before scaling.

Results

Three smoke attempts on L4, all OOMed at the same point: a 4.06 GiB allocation inside the autograd backward pass, ≈ 3 min 30 s into the run. The allocation size was identical across attempts despite progressive shrinking of the config:

Attempt	G_1	G_2	max_turn_tokens	Outcome
1 (50825d3)	4	2	400	OOM at backward, 4.06 GiB
2 (59f4d81)	2	2	200	OOM at backward, 4.06 GiB
3 (f735d1d)	2	2	100	OOM at backward, 4.06 GiB
4 (8b94cc7)	2	2	200	OOM at backward, 4.06 GiB (prompt trimmed)

Trimming the Multi-Turn Protocol prose from the prompt freed ≈ 300 MiB at peak (free memory 2.98 → 3.29 GiB) but did not move the 4.06 GiB allocation. Generation, sandbox execution, advantage computation, tool/loss-mask assembly, prompt-completion concatenation, and logp computation all completed — the override reaches `loss.backward()` on every attempt before failing.

Interpretation

The plumbing is sound; the structural issue is the loss layout. The 4.06 GiB allocation is the logits gradient tensor at backward, shape $(B, T_{\text{completion}}, V)$ with $V = 151{,}936$. The branched-N=2 setup makes both factors larger than single-turn:

- Per-row sequence is now [prompt | turn1 | tool | turn2], with the prompt itself enlarged by the multi-turn protocol description, scenarios, and env code (single-turn prompts were ≈ 500 tokens; branched prompts approach 1500–2000).
- The completion now spans turn-1 + tool result + turn-2 rather than a single turn.
- Backward retains activations across the full sequence at every transformer layer (36 layers, $H = 2048$); attention activations grow super-linearly in T.
- All $G_1 G_2$ trajectories run in one forward+backward graph, so memory peaks as a sum, not a max.

The single-turn G=16 baseline fit because per-row $T \approx 900$ and the prompt was short. Branched G=4 does not fit because per-row $T \approx 2500$ and the multi-turn prompt is large.

Hypotheses / next steps

The next task is *Option C: per-turn loss decomposition*, recorded in detail in the tracker task `nontrivial-envs/per-turn-loss-decomposition`.

1. **Split the forward into two passes.** Turn-1 logps need only [prompt | turn1] (G_1 short forwards, no G_2 redundancy). Turn-2 logps still need [prompt | turn1 | tool | turn2] but use `logits_to_keep` so the LM head only materializes (B, T_{turn2}, V) rather than the full completion span.
2. **Backward each pass separately.** Two sequential `.backward()` calls with `retain_graph=False` accumulate gradients into the same parameters; peak memory becomes max of the two passes rather than their sum.
3. **Custom `_compute_loss`.** Re-implement the DAPO loss segment-by-segment (≈ 30 lines) since TRL's parent expects one batched advantage tensor. Use the additive decomposition $\mathcal{L}_{ij} = \mathcal{L}_{ij}^{(1)} + \mathcal{L}_{ij}^{(2)}$ with the G_2 scaling on turn-1 to match the original sum.
4. **Optionally micro-batch turn-2** below $G_1 G_2$ to bound peak further if production sizing ($G_1 = G_2 = 4$, $T = 400$) still OOMs after the split.

Memory analysis (back-of-envelope, dominant logits-grad term only, at production sizing $G_1 = G_2 = 4$, $T_1 = T_2 = 400$, $T_t = 100$): current implementation $\approx 4.4\,\text{GiB}$; Option C peak $\approx \max(0.49, 1.94) = 1.94\,\text{GiB}$. With turn-2 micro-batching at $b = 4$, peak drops to $\approx 0.49\,\text{GiB}$.

Open questions

- Will the segmented forward correctly reproduce the concatenated loss numerically? A CPU unit test on a fixed input is in the task acceptance criteria; the math decomposes cleanly because tool tokens have $m_t = 0$, but tokenization-seam offsets between [prompt | turn1] and the full chat could shift boundaries and need careful handling via `apply_chat_template` on the full message list with index tracking.
- Does the additional forward (turn-1-only pass) cost more wall-time than it saves via the G_2-fold turn-1 redundancy reduction? Expected to be roughly neutral; confirm empirically.
- Generalization to $N \geq 3$: each turn gets its own forward+backward at $\prod_{k<n} G_k$ unique sequences. The pattern composes but compute grows multiplicatively per added turn.

Reproduce

Smoke config at `configs/smoke_test_grpo_branched.yaml`; run via `python scripts/09_run_loop.py --config configs/smoke_test_grpo_branched.yaml`. The OOM is reproducible at `8b94cc7` on a single L4 (24 GB).